

...I need to add library dependencies with
echo "Library dependencies"...



To automate finding, extracting, and adding the required shared libraries from your toolchain's **sysroot** into a target directory (like a rootfs or firmware folder), you can use the following `.sh` script.

This script parses the output of `readelf` for the **program interpreter** (dynamic linker) and **shared libraries**, locates them inside your sysroot, and copies them to your target directory.

bash

```
#!/bin/bash
set -e

# =====
# CONFIGURATION
# =====
SYSROOT="/opt/toolchain/sysroot"           # Path to your toolchain
TARGET_DIR="./rootfs"                     # Where you want to copy
BINARY="bin/busybox"                      # The binary you are checking
CROSS_COMPILE="aarch64-linux-gnu-"        # Your toolchain prefix

# Create target lib directories if they don't exist
mkdir -p "${TARGET_DIR}/lib"
mkdir -p "${TARGET_DIR}/usr/lib"

echo "=== Processing Binary: ${BINARY} ==="

# =====
# 1. FIND & COPY THE PROGRAM INTERPRETER (DYNAMIC LINKER)
# =====
echo "Finding program interpreter..."
INTERPRETER=$((${CROSS_COMPILE}readelf -l "$BINARY" | grep "program

if [ -n "$INTERPRETER" ]; then
    echo "Found interpreter: $INTERPRETER"
    # Find the interpreter inside the sysroot
```

```

SYSROOT_INTERPRETER="${SYSROOT}${INTERPRETER}"

if [ -f "$SYSROOT_INTERPRETER" ]; then
    # Determine target directory structure match
    DEST_DIR="${TARGET_DIR}$(dirname "$INTERPRETER")"
    mkdir -p "$DEST_DIR"
    cp -v -d "$SYSROOT_INTERPRETER" "$DEST_DIR/"
else
    echo "Warning: Interpreter not found at ${SYSROOT_INTERPRETER}"
fi

# =====
# 2. FIND & COPY SHARED LIBRARIES
# =====
echo -e "\nFinding and copying shared library dependencies..."
# Extract library names from readelf output
LIBS=$((${CROSS_COMPILE}readelf -d "$BINARY" | grep "Shared library"

for LIB in $LIBS; do
    echo "Processing: $LIB"

    # Search for the library file inside the sysroot's lib directory
    LIB_PATH=$(find "$SYSROOT" -name "$LIB" -print -quit)

    if [ -n "$LIB_PATH" ]; then
        # Determine if it belongs in /lib or /usr/lib based on sys
        if [[ "$LIB_PATH" == */usr/lib/* ]]; then
            cp -v -d "$LIB_PATH" "${TARGET_DIR}/usr/lib/"
        else
            cp -v -d "$LIB_PATH" "${TARGET_DIR}/lib/"
        fi
    else
        echo "ERROR: Shared library $LIB not found in sysroot!"
    fi
done

echo -e "\nDone! All dependencies copied to ${TARGET_DIR}."

```

Use code with caution.

Why this script works:

Why this script works.

- `readelf -l` vs `-a`: Using `-l` (program headers) isolates the **program interpreter** line cleaner, and `-d` (dynamic section) isolates **Shared libraries** without dumping the entire file data, making the text processing significantly faster.
- `cp -v -d`: The `-d` flag is critical. It preserves symlinks (e.g., `libc.so.6` -> `libc-2.31.so`) instead of expanding them into massive duplicate files, which saves space on your target filesystem.
- **Dynamic Paths**: It dynamically checks if the file came from `/usr/lib` or `/lib` inside your sysroot and mirrors that structure in your output folder.

To make sure this handles deep dependencies, let me know:

- Do the shared libraries you are copying **have their own dependencies** that also need to be checked recursively?
- What is the **exact folder structure** of your target root file system (e.g., are all libs combined into one folder, or split into `/lib` and `/usr/lib`)?

what is the -n for

In a shell script, `-n` is a conditional operator used to check if a text string is not empty.

It stands for "**not zero length**" (nonzero).

How it works in the script:

In the line `if [ -n "$INTERPRETER" ]; then`, it evaluates like this:

- **If `$INTERPRETER` contains text**: The condition is true, and the script proceeds to look for and copy the dynamic linker.
- **If `$INTERPRETER` is completely empty**: The condition is false, and the script skips the copying process so it doesn't crash on an empty file path.

The opposite operator: `-z`

If you ever want to check if a string **is** empty, you use `-z` (which stands for "zero length").

bash

```
if [ -z "$INTERPRETER" ]; then
    echo "The string is empty!"
fi
```

Use code with caution.

A critical rule for `[ -n ]`

Always wrap your variable in **double quotes** (e.g., `"$INTERPRETER"`). If you forget the quotes and the variable is empty, the shell will evaluate the command as `if [ -n ]; then`. Because `-n` is left hanging without an argument, Bash will incorrectly evaluate it as true, which can cause bugs in your script.

Would you like me to show you how to expand that script to **recursively** scan those newly copied `.so` files for *their* dependencies too, or are you ready to test it out?